AI ENGINEER / AGENTS & AUTOMATION

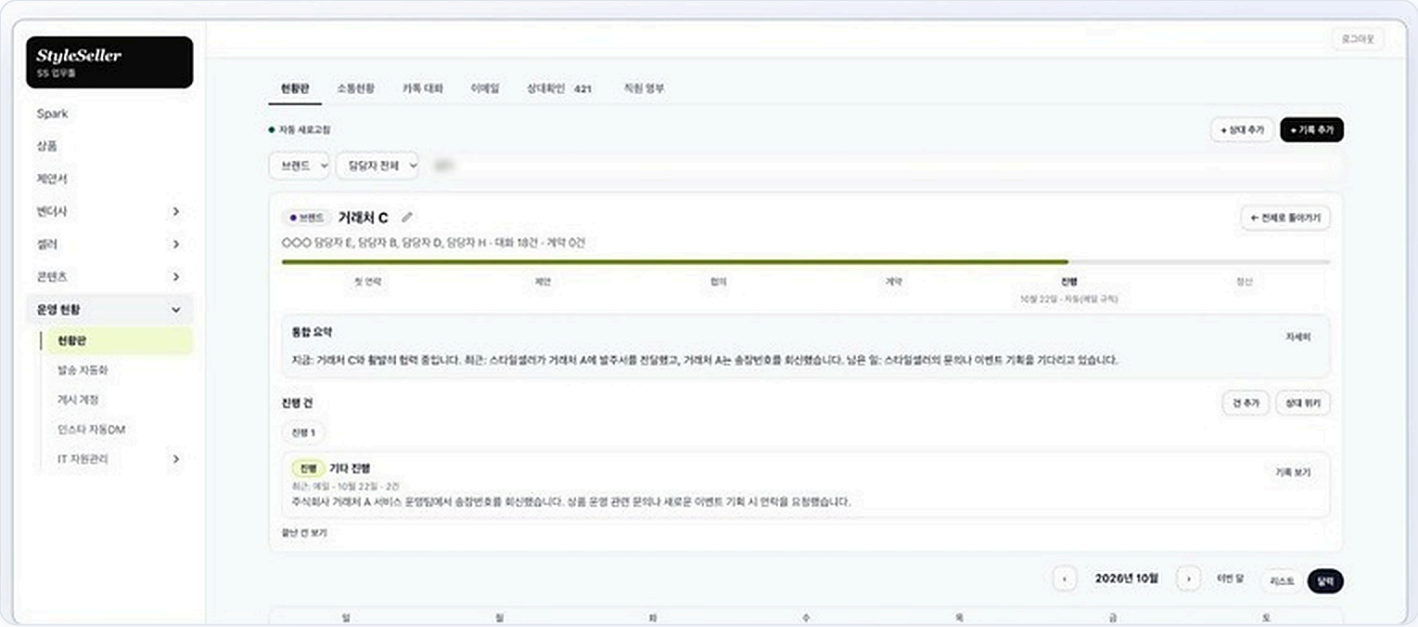
Portfolio

Turning everyday business work into AI systems people use.

Jinhee Mok

AI & Automation Engineer · Design, deployment and operations

At StyleSeller, I build tools for partner support, shared knowledge and day-to-day operations. I deploy them to real staff PCs, investigate failures and improve the workflows after launch.



Production dashboard. Identifying details are blurred.

01 / page 4

Status Board

One place to follow a business partner's conversations, orders and deal stage.

02 / page 6

Content Autopilot

A Reel workflow with predictable stages, a reviewable preview and controlled publishing.

03 / page 8

Vendor CS Agent

Tagged KakaoTalk questions answered against partner records, with delivery checked on the staff Mac.

04 / page 10

Agent Platform · Spark

Work orders, agent delegation and tool execution with controls enforced by the server.

05 / page 14

Knowledge Hub · Email Agent

Shared search with evidence checks before email drafts reach the assigned staff member.

06 / page 16

Internal Work Tool

Reliable pricing, reviewable catalog candidates and controlled personalized KakaoTalk sends.

07 / page 18

Staff PC Fleet

Shipping and operating tools on staff Macs and Windows PCs, then staying responsible after installation.

Selected work for Forward Deployed Software Engineer roles.

jinheemok815@gmail.com · jin-tonix.github.io/en/

Jinhee Mok

jinheemok815@gmail.com · +82 10-9030-2888
jin-tonix.github.io
Yeoksam-dong, Gangnam-gu, Seoul

AI & Automation Engineer · Design to deployment and operations

Resume

Summary

I build AI agents and automation for repetitive work, and stay involved after launch. Since joining StyleSeller in November 2025, I have delivered tools for partner support, email replies, knowledge search and staff PCs. Before software, I worked in daycare operations and English teaching. I use Claude Code for implementation while defining the scope and reviewing the results myself.

Experience

StyleSeller · AI & Automation Project Lead (PL)

2025.11–present

- Designed, built and operated partner-support agents, Spark and work automation.
- Built a shared knowledge hub and claim-by-claim checks for email drafts.
- Improved skill selection: 119/132 correct (90.15%) after correcting evaluation errors.
- Deployed tools to eleven staff PCs; operated GCP, Docker and self-hosted databases.

We:Review · AI Development Intern

2025.08–2025.10

Built an OCR-to-structured-data receipt pipeline with PaddleOCR, DBSCAN, Qwen2.5-7B and FastAPI. Took part in deployment: review accuracy 70% → 95%, review time reduced by 85%.

Jeongsang · Pots&Pans · Montekids · English Teacher

2018.08–2024.02

Designed lessons for ages 5–15, prepared materials and individual learning plans, and held parent consultations.

Jeonwon Daycare / After-School · Operations Manager

2015.03–2018.02

Hired teachers, planned the curriculum and events, worked with parents and the steering committee, and managed daily operations as acting director.

Education

Cheongju University · Law · 2008–2012

Academic Credit Bank System · Business Administration · 2021–2023

Training

Himedia · Full-Stack (960h) · 2024.04–10

Himedia · Advanced AI · 2024.10–12

Korea IT Business Promotion Association · Generative AI · 2025.05–07

Skills

AI/LLM · Gemini, LiteLLM, LightRAG, MCP, tool calling, evaluation design

Backend · Python, FastAPI, TypeScript, Node.js, PostgreSQL

Infrastructure · GCP, Docker, NGINX, Vercel, GitHub Actions, n8n

At a Glance

Seven systems I built and operate at StyleSeller, plus one follow-up improvement. Each case explains the problem, the implementation, the choices I made and the limits that remain. Figures carry their measurement dates.

Status Board Page 4

One place to follow a business partner's conversations, orders and deal stage.

Content Autopilot Page 6

A Reel workflow with predictable stages, a reviewable preview and controlled publishing.

Vendor CS Agent Page 8

Tagged KakaoTalk questions answered against partner records, with delivery checked on the staff Mac.

Agent Platform · Spark Page 10

Work orders, agent delegation and tool execution with controls enforced by the server.

Jev Skill Router Upgrade · Spark follow-up Page 13

Improved skill selection and corrected the evaluation. 119/132 correct; estimated 90% lower cost from reducing Gemini input.

Knowledge Hub · Email Agent Page 14

Shared search with evidence checks before email drafts reach the assigned staff member.

Internal Work Tool Page 16

Reliable pricing, reviewable catalog candidates and controlled personalized KakaoTalk sends.

Staff PC Fleet Page 18

Shipping and operating tools on staff Macs and Windows PCs, then staying responsible after installation.

Status Board · Operations Ledger

One place to follow a business partner's conversations, orders and deal stage.

Period	Built October 19–23, 2026 · In production
Role	I designed the ledger and matching rules, built the intake API, email collector and six UI tabs, and verified the database behavior and production screens. I used Claude Code for implementation, while setting the scope and reviewing the results myself.
Stack	React · Python · PostgreSQL/RLS · Gemini via LiteLLM

Background

Staff followed the same deal across KakaoTalk rooms, email, Slack and emailed order sheets. I built a dashboard that brings these records onto a calendar for each business partner and shows deal stages across hundreds of partners.

How it works

1. Collect: KakaoTalk runs three times a day; email every five minutes; Slack uses a connector. Source records enter a local queue before delivery.
2. Prepare: Code filters newsletters and masks buyer details in order-sheet columns. The LLM extracts summaries; cited phrases are checked against the source.
3. Link and store: An idempotent ledger stores each record once. Ordered rules link partners by thread, registration number, room and contact details. Conflicts remain unclassified.
4. Review and follow up: Staff resolve unclear matches and stages. An 18:05 batch writes 3–6-line summaries for up to around 500 partners, stopping at its daily cost limit.

> **Scale and evidence.** Communication Status share by stage: first contact 13.1%, proposal 33.8%, negotiation 2.6%, contract 4.5%, in progress 46.0%, settlement 0% (2026-10-24) · Roughly 100–200 records per weekday, hundreds awaiting business partner matching (2026-10-24)

Incidents and fixes

All 107 summaries rejected (2026-10-23). The application role lacked execute permission on the summary-saving function. All 107 saves failed with db_error. Granting execute access to that function restored the path.

Too many 'in progress' proposals in the rehearsal (2026-10-23). Broad words such as shipping and inventory triggered false deal signals. I kept only actual deal terms and removed negative phrases such as sales halted and order inquiry before matching.

Shared chatbot shown as the contact person (2026-10-23). A shared chatbot appeared among the five most active room speakers and became the contact for 64 partners. I now use company email accounts, falling back to the account-profile contact.

Technical highlights

1. Retries preserve staff edits. An upsert on (source, source_ref) ignores unchanged content and protects staff-edited records. Duplicate intake does not bump sequence numbers or timestamps.

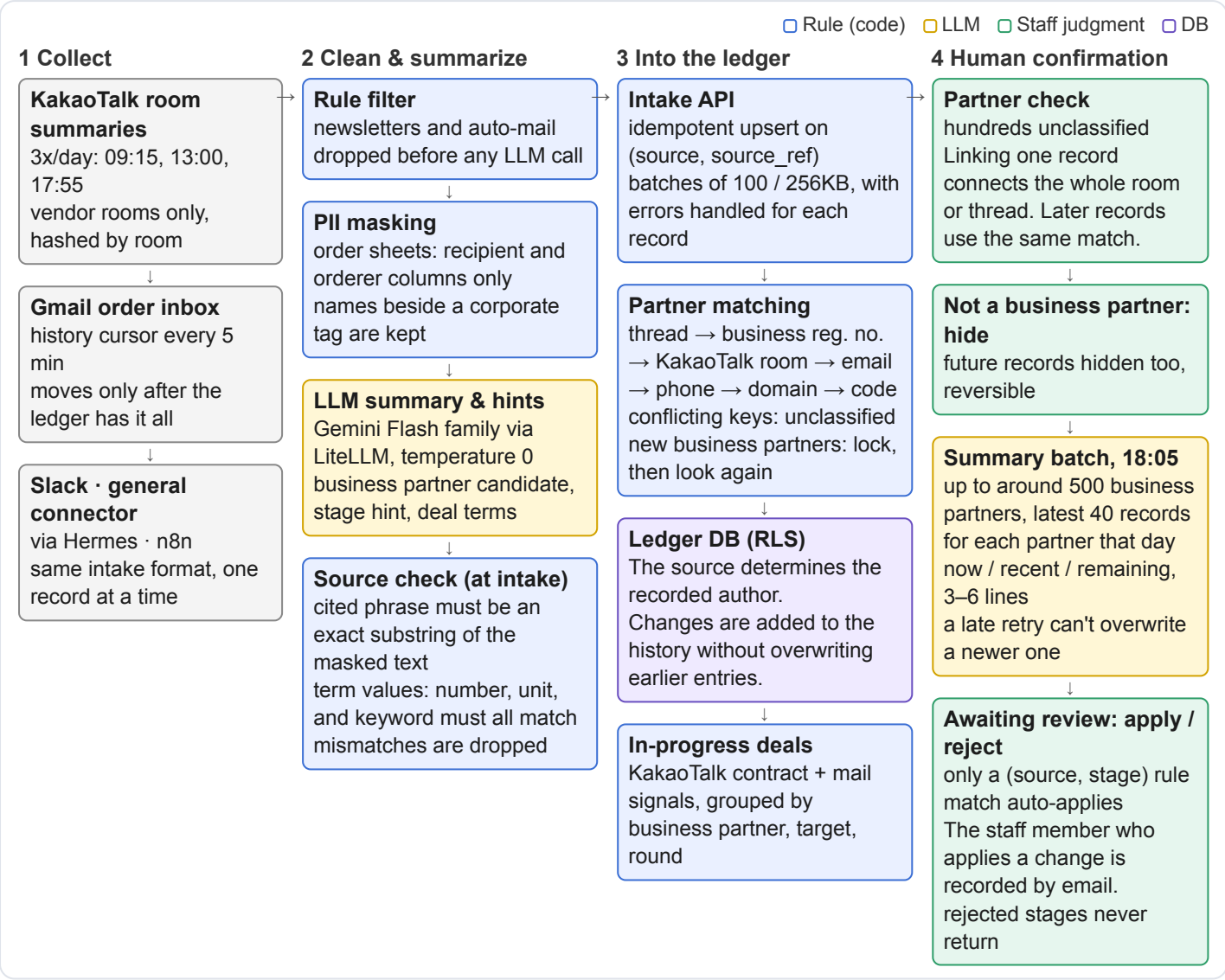
2. Authorship comes from the source. The server rejects the caller's author value. Database triggers set the staff-edit flag; hidden records cannot be restored by an ordinary update.

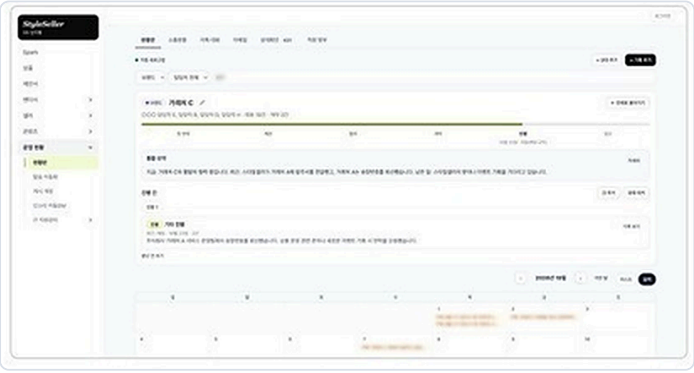
I enable automatic stage rules one at a time; model confidence is not calibrated, so it does not decide a stage. Python, TypeScript and SQL share normalization examples. CI verifies content using PGlite as the real app role; UI tests run separately.

> The six tabs, three intake channels and daily summary batch were checked on October 24. Intake was about 100–200 records per weekday. Staff acceptance of suggested stages has not been measured; the server currently trusts the intake-side evidence checks.

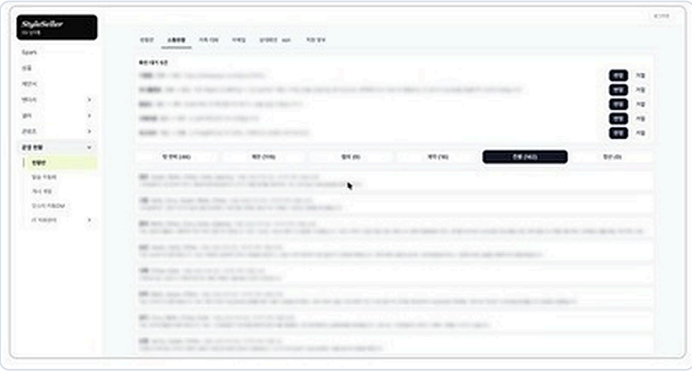
Status Board · Operations Ledger

Architecture





Partner calendar · conversations, email and order sheets in date order.



Deal-stage board · staff apply or reject pending suggestions.

Technical highlights

3. Conflicting keys stay unclassified. Matching follows a fixed order. Shared public domains are excluded; creating a partner takes a lock and rechecks to avoid duplicates.

4. Extraction needs source evidence. Cited phrases must occur in the masked source. Deal numbers, units and keywords must match; buyer columns are masked while product details remain.

5. Costs and cursors survive concurrency. Cost checking and reservation share one SQL statement. The email cursor advances only after delivery succeeds; repeated failures are quarantined and reported.

Reels Content Autopilot

A Reel workflow with predictable stages, a reviewable preview and controlled publishing.

Period	October 2026 onward · Render path verified; publishing staged
Role	I designed the state machine and publishing defaults, built the orchestrator, database publishing slots and QA gate, and checked the production render path and restart behavior.
Stack	Paperclip · Hermes · Python · HyperFrames · Drive · PostgreSQL

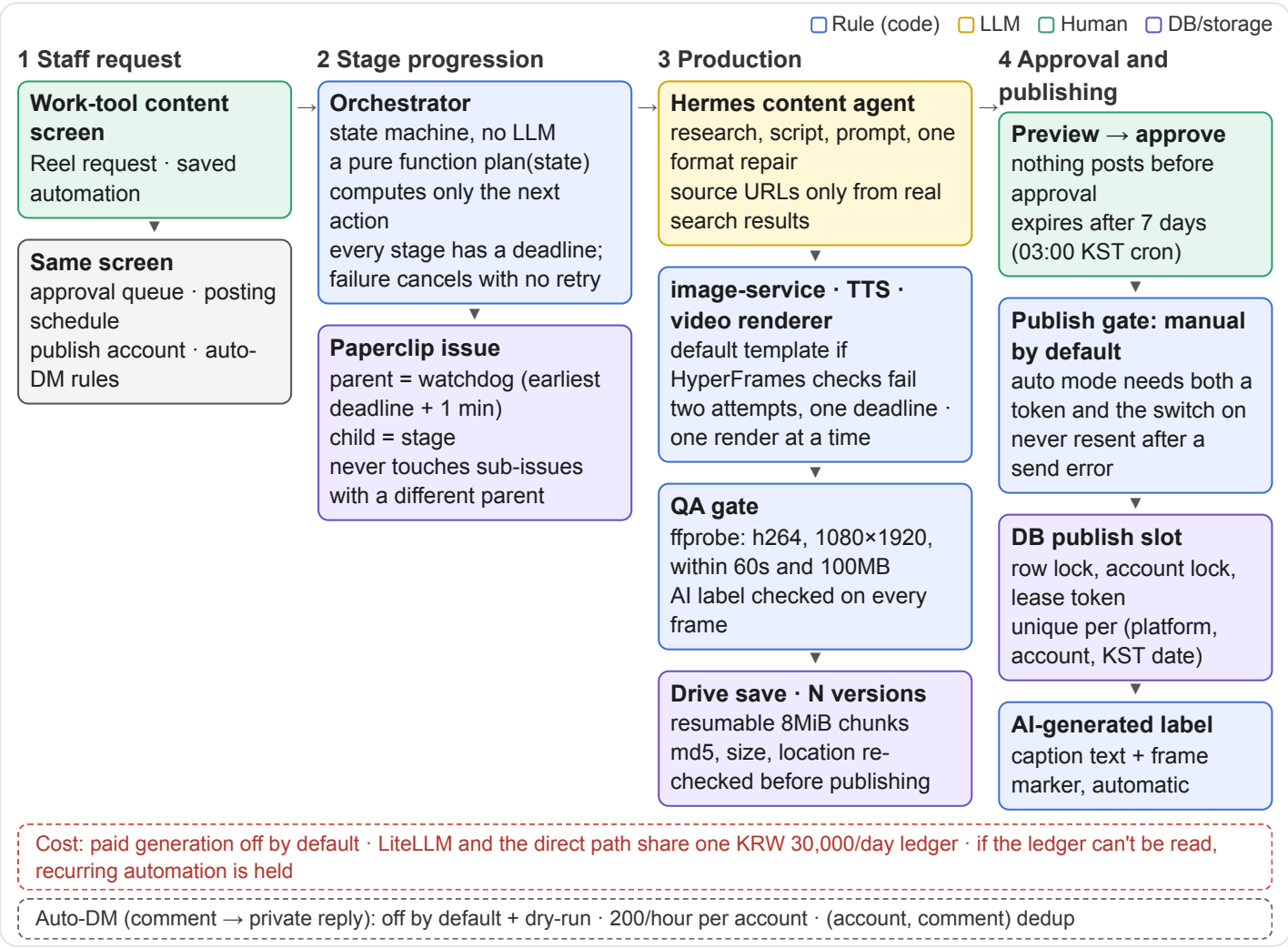
Background

Generating a video costs money, and publishing makes it public. I built a workflow where staff request a Reel, inspect the rendered video and approve it in the work tool. Code controls the stages, while an agent prepares research, scripts and prompts.

How it works

1. Request: Staff submit a Reel request or start a saved automation. Recurring automation simulates execution by default; paid generation is off.
2. Plan and produce: A state machine selects the next action. Hermes prepares research and a script; the renderer produces a vertical MP4 from a template.
3. Check and approve: QA checks format, dimensions, duration, size and AI markers. A checksum identifies the saved file. Staff approve the preview; approval expires after seven days.
4. Publish: Publishing is manual by default. Automated publishing requires a token and switch, a database slot and a final file check. Unknown outcomes are reconciled before another attempt.

Architecture



> **Scale and evidence.** About 122k lines across 3 services, 617 test files (orchestrator 135, Hermes agent 396, video renderer 86) · Production canary verified through saving a 1080×1920 MP4 to Drive (2026-10) · 0 live SNS posts from the new production path. One post per account per day is enforced by the DB; the publish gate caps at 10 per day (2026-10-24)

Defaults and controls

Posting. The default is manual. Auto-posting turns on only when both the token and the switch are present. A partial unique index on (platform, account, KST date) blocks more than 1 post per account per day. If duplicates already exist, the migration stops first.

Paid generation and recurring automations. The paid backend defaults to False. Recurring automations default to dry-run and are released only when the value is exactly "false". Before starting, they read the company's ₩30,000-per-day ledger, and hold if it can't be read. The cap is clamped with min() so env can't raise it.

Instagram auto-DM. By default the enable switch is off and dry-run is on. The limit is 200 per account per hour, and invalid config values are read toward the safe side. A unique index on (account, comment) stops a single comment from getting two DMs.

Technical highlights

1. Code chooses the next stage. A planner returns actions and an executor applies them. Deadlines wake the parent issue; a rerun does not create a second pipeline or touch another parent's work.

2. Repairs are bounded. JSON errors get one repair call with up to 8 errors of 120 characters each. A failed motion plan gets one default-template render within the original deadline; paid or public actions need another human decision.

3. Sources must come from real search. Cited URLs must appear in tool results or the supplied input. Model-written evidence fields are discarded; unsupported search claims fail verification.

4. Unknown publishing outcomes are reconciled. A marker is written before the POST. Uncertain results stay pending; saved submission IDs are checked after restart instead of publishing again.

5. The database allocates publishing slots. Row and account locks issue a 30–900-second lease after checking the cap. A unique index enforces one post per account per day across processes.

6. Cost paths share one ledger. LiteLLM and direct Vertex calls write to the same daily key. A ledger-read failure holds recurring automation starts; individual calls follow a separate policy.

7. Approval is tied to the approved file. Before posting, recheck the Drive file type, size, checksum and location. QA checks H.264, 1080×1920, 60 seconds, 100MB and required AI markers. Uploads use 8MiB chunks and resume on HTTP 308.

> The production trial generated a 1080×1920 MP4 and saved it to Drive. As of October 24, the new generation path had published no live posts, so it had no measured reach results. Existing videos used a separate publishing path. Product-price and bundle checks still rely on prompts and human preview; I would add database checks before expanding rollout.

Design choices

- A state machine with no LLM drives stage progression. The LLM only writes research, scripts, and prompts. If the model picked the next stage, every rerun would give a different result.
- Apart from one format repair and one re-render with the default template, it closes on failure instead of retrying automatically. Anything that costs money or goes public, a human clicks again.
- Posting defaults to manual, and the daily cap lives in a DB index, not in app config.

A daily cleanup rejects expired approvals, cancels their issues and closes the rows even if cancellation fails. Registering a new Instagram token still requires developer setup.

Vendor CS Agent

StyleSeller · Private repositories

Tagged KakaoTalk questions answered against partner records, with delivery checked on the staff Mac.

Period	June 2026 onward · In production
Role	I designed the intake and partner gates, built the agent and local send path, added evidence checks and delivery confirmation, and investigated failures on real Macs and Windows PCs.
Stack	Swift kakaocli fork · Python · Gemini · LightRAG · Sheets/Drive

Background

Partner conversations live on staff computers, but the agent needs shared records to answer consistently. I connected local collection, a server ledger and the knowledge hub, then returned replies through a staff Mac. Since September 2, the agent answers tagged questions without follow-up questions or a human handoff step. Since September 2, it answers tagged questions without follow-up questions or a human handoff step. Code checks amounts, percentages and brands against retrieved evidence before delivery.

How it works

1. On Mac, kakaocli reads only what comes after the last logId in the KakaoTalk DB. On Windows, a read-only collector uploads it. It moves on only after the server answers with 2xx.
2. The server drops the same message if it arrives again within 15 seconds. It stores records in per-partner ledgers, Sheets, and Drive, and twice a day (09:30, 18:10) compiles them into a wiki that goes into the knowledge hub.
3. The business partner ID or name in the room name is checked against the work tool's list of business partners to serve. If no business partner is found on the list, it does not answer.
4. It accepts only questions tagged @회사_AI. The regex only detects the tag; the model does the search and writes the answer. Other business partners' names are masked as [다른 벤더].
5. Amounts, percentages, and brands in the draft are checked against the evidence, and on a mismatch it searches again and rewrites. The reply is sent from a staff Mac, and delivery is confirmed by whether the KakaoTalk DB count went up.

Incidents and fixes

A regex blocked questions before model assessment (until 2026-08-30). Some messages were silently blocked without any LLM judgment. The regex meant as a trigger was also doing the blocking. I cut the regex down to tag detection only and moved the decision on whether to answer to the model.

Background KakaoTalk calls hung (2026-08-17–18). On macOS 15, when the KakaoTalk window was in the background, Accessibility (AX) calls ignored their timeout and hung. 31 of 147 sends (21%) failed permanently. I moved the calls to a separate thread and put a 3-second deadline on them with a semaphore (past that, the wait is cut off). Since 09-08 it also re-checks delivery after sending.

Flood of past conversations (2026-09-16). Three Windows PCs uploaded 2,500 records in 50 minutes, the Sheets API was blocked with 429, and rows for the 2 people installed later stayed empty for over an hour. An old collector build had uploaded entire past conversations. I made the server drop records older than a cutoff date and answer 200. Answering with 5xx makes the collector upload the same records again.

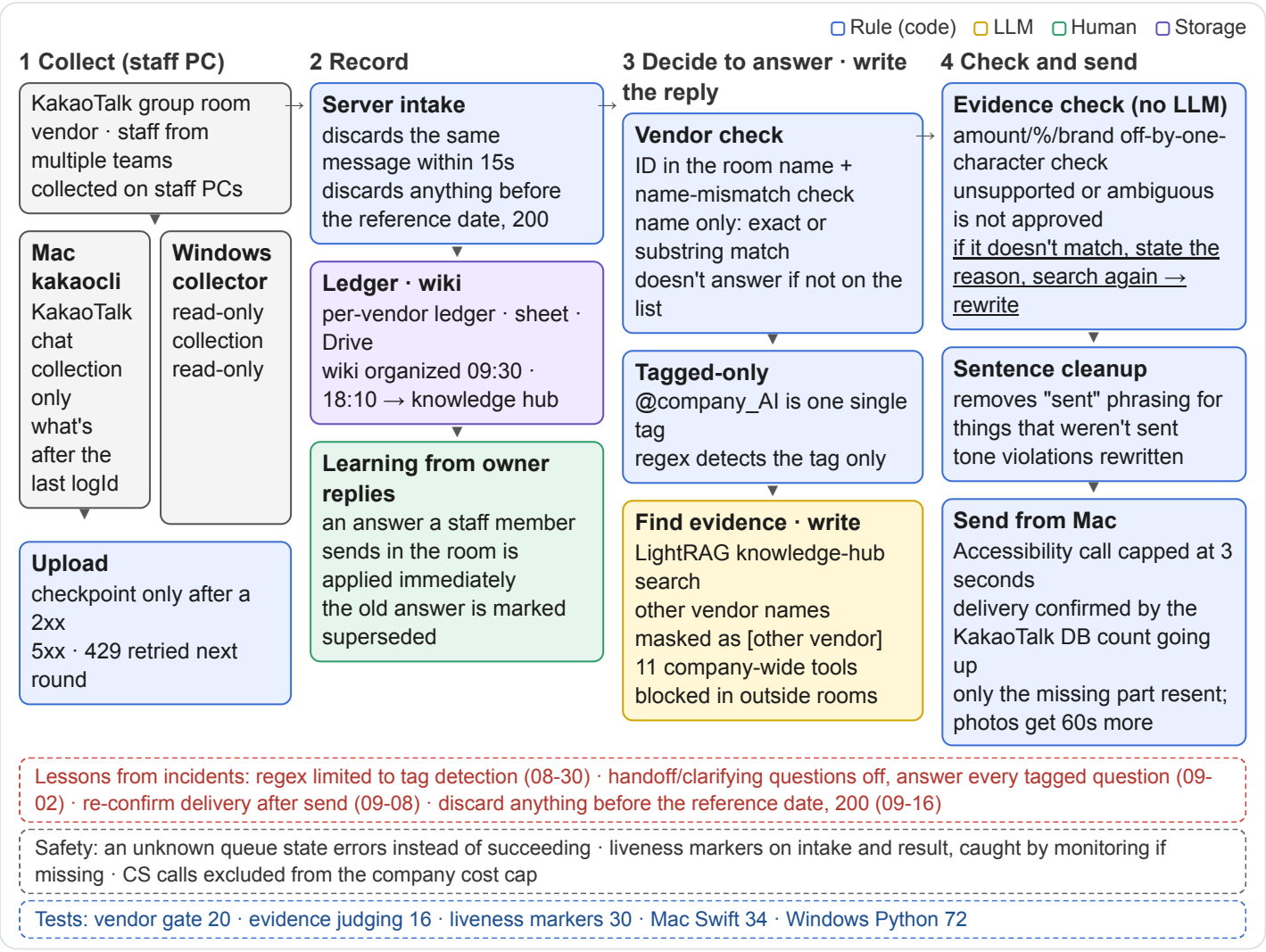
> **Scale and evidence.** Vendor agent code, tests, and config: 61K lines total, 600 commits, 107 test files (2026-09-23) · CS Copilot extension and local sender: 631 commits, 20,596 lines, 56 tests (2026-09-23) · Tests: business partner gate 20, evidence judgment 16, liveness markers 30, Mac Swift 34, Windows Python 72

Direct staff replies are learned immediately; newer answers supersede earlier ones and lower their search priority.

> Processing writes start and result markers so monitoring can detect a tagged question that never reaches an outcome. The Windows collector still updates by reinstalling. Vendor and influencer support calls are excluded from the company daily cost cap. I would include a collector updater and server cutoff date from the first release.

Vendor CS Agent

Architecture



Technical highlights

1. Require a clear partner match. Room IDs and names must agree. Name conflicts at similarity 0.80 are blocked. Common suffixes scored 0.81–0.88, so I removed fuzzy matching; ambiguous rooms receive no answer.

2. Deduplicate without losing room changes. One message arrived 51 times, amid 170–200 incoming messages per minute. A 15-second key includes room, message, unread count and sender direction.

3. Check numbers and brands in code. Normalize amounts and percentages (for example, 3만원 becomes 30000 KRW), compare them with evidence and flag near-matching brands. Missing or ambiguous evidence triggers another search and rewrite.

4. Do not claim an action that never ran. Remove completed-send wording when no send occurred. Retries read the real queue state; only failed items are re-queued.

5. Keep external rooms within scope. Other partners' names are masked. Eleven company-wide Spark tools are blocked, and the server supplies the partner identity for product lookup.

6. Confirm delivery locally. Count new text and photos in the KakaoTalk database. Retry only missing parts, allow extra time for photos, then report completion.

7. Acknowledge and monitor the processing path. Uploads skip rejected 422/413 items; 429/5xx stop the batch for retry. Start and result markers detect tagged questions that never reach an outcome; this path has 30 tests.

The earlier human-handoff branch remains disabled and covered by tests.

Spark Agent Platform · Spark

Work orders, agent delegation and tool execution with controls enforced by the server.

Period	May 2026 onward · Spark and Work OS from July
Role	I designed work orders and delegation limits, integrated shared skills and tools, built single-use approvals and cost controls, and wrote evaluation cases and deployment checks.
Stack	Paperclip/Express · Hermes · PostgreSQL · FastAPI · LiteLLM · Langfuse

Background

An agent can change the meaning of a request as it delegates work, and a tool can write data, spend money or publish content. I forked Paperclip and added Spark, Work OS and company services, with execution controls in server code. Staff use Spark to query four business databases and hand work to the CEO agent.

How it works

1. Requests from Spark, Slack, and the work tool all go to the CEO agent. Only structured requests from the work tool go straight to the content orchestrator, and they still leave a work order record.
2. The CEO writes work order v1. It holds the original text as an attachment with its sha256, the goal, completion criteria, constraints, and whether approval is required.
3. An overall orchestrator reporting directly to the CEO (the Hermes dispatcher) assigns it to one of at most 10 department heads, such as the Supervisor and fix agents. The server caps delegation depth at 2 and delegations per work order at 5, and blocks A→B→A bounce-backs. Turning it off falls back to direct delegation by the CEO. Work that goes to the content department head is handed on again to the content orchestrator, a separate service.
4. BM25 search pulls candidates from all shared skills, the judge model Jev checks gating and fit, and Gemini makes the final pick of one skill. If Jev fails or times out, it is skipped and Gemini picks directly from the full list.
5. Tools run through a 6-stage gate and human approval. Publishing goes through the Publisher gateway (a single service dedicated to publishing). The default is manual, and the DB enforces one post per account per day.

Incidents and fixes

Spark shut down entirely (2026-10-24). A new Spark tool was missing its DB registration row, so the boot check failed and every Spark call was rejected. I built it to block when the check fails, and it did exactly that. After recovering with a restart, I added a rule that every tool's registration row ships as a migration, plus a regression test.

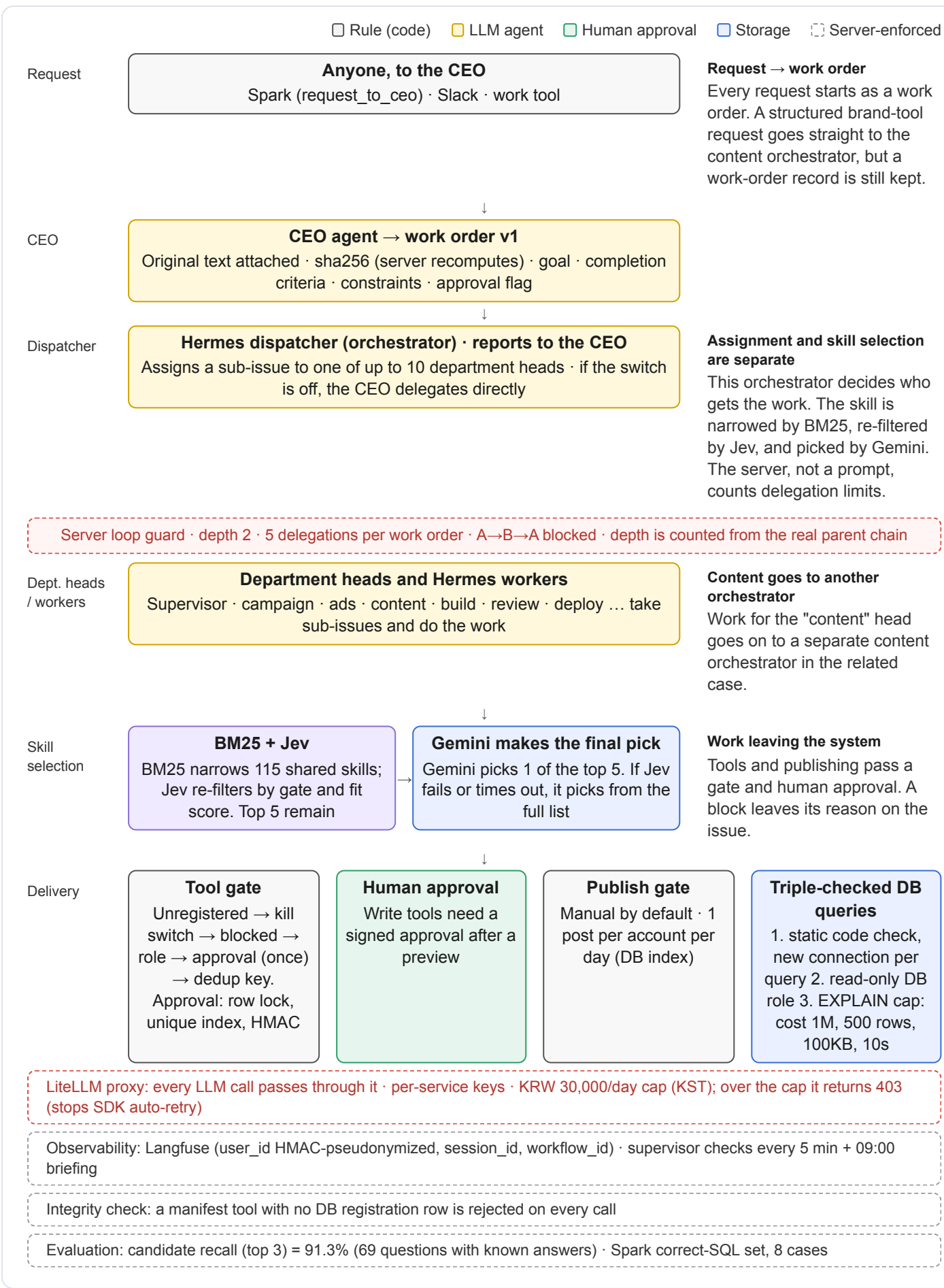
Reclaiming a job mid-render (2026-10-24). The server reaper treated a video job running past 5 minutes as stalled and reclaimed it, and the result that arrived 7 min 40 s later was rejected with 409. I made the job send a heartbeat every 120 seconds and stop once the issue deadline passes. Jobs that are really stuck still get reclaimed. In production I confirmed heartbeats received (200) and 0 reclaims.

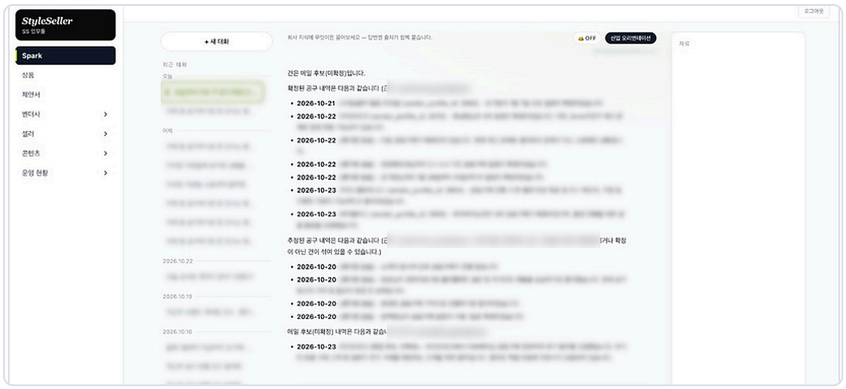
Jev call path blocked (2026-10-23). The existing Jev call path was blocked. I added an adapter that only matches the response format and sent calls to a lightweight model. The code in the 3 routers stayed the same.

> **Scale and evidence.** About 5,000 of my own commits (excluding merges and upstream), about 2,200 test files (2026-10-24) · 109 shared skills (October 24, 2026), 66 trigger eval files (1,282 sentences), 67 tool manifests

After two Jev failures, a 300-second circuit breaker skips that step. Rollout groups use the first eight SHA-256 characters. The server counts delegation depth from the real parent chain, rejecting forged depth and a sixth delegation.

Architecture





Spark in production. Question titles and partner names are blurred; answers identify their source tool.

Approvals refer to exactly what staff reviewed

A write approval signs the input and preview hashes, requester, run ID and expiry. A row lock and unique index prevent reuse; approval is consumed even when execution fails. A concurrency test confirmed one execution from ten simultaneous requests. Hashing the original request also lets the server detect changes across the delegation chain.

Read-only database access needs several controls

Spark uses a new connection for each query, filters dangerous functions and writing CTEs, rejects EXPLAIN costs above 1,000,000, and limits execution to ten seconds and results to 500 rows or 100KB. Reusing a connection could carry changed session settings into another person's query. Audit logs retain the SQL hash.

An outage exposed a missing deployment check

On October 24, a new tool lacked its database registration row. The startup check rejected all Spark calls. After recovery, I required registration rows to ship as migrations and added a regression test. A separate long render was reclaimed as stalled; a 120-second heartbeat kept active renders alive while preserving recovery for truly stalled jobs.

Cost and observability

Model calls pass through LiteLLM with service-specific keys. The daily paid cap returns a non-retryable response rather than a rate-limit response that the SDK would retry automatically. Langfuse receives a pseudonymized user ID, session ID and workflow ID. These records let me connect a user request to the tools, model calls and operational checks that handled it.

Measured scope

The October 24 snapshot contained 109 shared skills, 67 tool manifests and 66 trigger-evaluation files. Candidate recall was 85.5% at top one and 91.3% at top three on 69 questions with known answers; median latency was about 1.09 seconds. This measures whether the correct candidate was present, not final answer accuracy. The following Jev case measures final skill selection separately.

Design choices and alternatives

- I chose a full index over per-role skill bundles. On a 57-case selection set both scored the same accuracy, 0.947 (p=1.0). But the bundle approach missed skills that only get used when called by name. The cost difference over 66 cases was 27 KRW.
- The dispatcher assigns work; BM25 shortlists candidates, Jev re-filters them, and Gemini picks the skill. If Jev fails, Gemini still picks directly from the full list, so work does not stop. The worker never picks the skill; Gemini always does.
- Controls sit where they still hold if the model ignores them. The loop guard, single-use approval, and paid cap all live in the server and DB.

> Next: Jev skill selection, evaluated separately from candidate recall.

Jev Skill Router Upgrade

Improving skill selection started with checking the evaluation itself.

I designed the test, integrated the router, investigated failures and chose the rollout settings. Hermes uses BM25 to narrow skills, Jev to check fit and Gemini to select the final skill. If Jev fails, Gemini selects from the full list.

90.15%
119 of 132 cases correct

115 → 5
Skills reviewed by Gemini

Rolled out
Hermes sub-agents

Corrections and re-measurement

Five of six flagged false positives were answer-key errors. Some apparent gains existed only on questions already used during tuning. A fake skill in the test folder caused another false conclusion; it did not exist in production. I removed it and reran both sets: 90 + 132 = 222 cases.

The skill catalog grew from 109 on October 24 to 115 at test time. The judge alone reached 45–53%; adding the LLM proposer on October 26 brought results into the 80s. Gains on tuned questions initially looked like +4.4 percentage points, versus +1.2 on new questions. After cleaning and re-measuring, those became zero and +1.5, so I stopped treating the tuned-set gain as evidence.

Evaluation stage	Production setting	Comparison setting
60 cases · answer-key errors present	83.3%	91.7%
90 cases · corrected answer key	88.9%	93.3%
Same 90 · fake skill removed	94.4%	94.4%
132 new cases · later reused	90.15%	91.7%

The rows use corrected labels and changing sets. They are not a clean before-and-after accuracy curve.

Rollout and cost reduction

The deployed router reduces Gemini review from all 115 skills to five. Published unit prices and the reduced input imply about 90% lower cost than the full-list approach; this is a calculated estimate. Jev returns probabilities rather than generated text, with a measured median response time of 360ms. All three production check questions passed before full rollout.

The estimated cost is about 1/10 of the full-list approach, with similar selection accuracy in this evaluation. I compared settings using saved answers, expanded experiments from 20 to 90 to 132 questions and recorded zero HTTP 429 errors. Before rollout I checked all three changed code locations, backed up configuration, restarted the server and verified its health.

I set the rollout criteria before testing: better selection on unseen questions without more wrong skill picks. A higher score alone was not enough to enable a setting.

What I held back

A comparison setting gained only 1.5 percentage points and sometimes chose a skill with fit as low as 0.22 despite high confidence. I left it off.

What the result means

The 132 questions were new when introduced, then reused in tuning. Some questions have multiple valid answers. Catalog changes caused 10 label errors (five in each set); automatic label checks remain unbuilt. The result does not guarantee production-wide accuracy.

RAG LightRAG Knowledge Hub · Email Agent

StyleSeller · Private repositories

Shared search with evidence checks before email drafts reach the assigned staff member.

Period	March 31, 2026 onward · In production
Role	I designed workspace registration, search planning and evidence checks, built the mail server and draft checkers, and handled rollout, incident investigation and recovery.
Stack	Python/FastAPI · LightRAG · Neo4j · pgvector · FlashRank · Gmail

Background

Vendor support, influencer support, Spark and partner matching need the same business knowledge. I built a shared LightRAG hub with 13 workspaces, each with an owner, channel and sensitivity level. For shared-mailbox inquiries, the email agent prepares a checked draft in the assigned person's Gmail drafts folder; staff send it.

How it works

1. An LLM planner (temperature 0) reads the workspace descriptions and data catalog and builds up to 12 queries. A plan that fails validation is discarded and the rule router takes over.
2. Code pins the search mode (mix, which uses vector and graph together) at each call site. Agent searches use top_k (number of results fetched) 8, and identical queries are cached for 60 seconds.
3. Evidence is judged by rules. Live values such as price, stock, and settlement are forbidden claims. Refund confirmation, contracts, and bulk sends get a human approval flag, and only the missing evidence items are searched once more.
4. Results are grouped by topic using rerank (the step that re-orders results) scores. Within a group, official corrections and the newest documents go first, and output alternates one result per group.
5. A checker and an LLM verifier compare up to 6 claims in the email draft against the evidence. If a claim has no evidence the draft is rewritten, and if the problem remains the draft is discarded.

Incidents and fixes

Missing rerank (2026-06-19~07-23). On 06-18 we switched to agentic RAG (the LLM plans the search). From the next day, the quality of CEO email drafts dropped. Tracing back through the commit history showed the rerank step had been dropped in the switch. I restored it on 07-23.

Search overload (07-10, 08-12). The influencer CS bot took the whole shared connection pool, so even partner-matching searches returned HTTP 500 errors (07-10). top_k 40 searches triggered Neo4j overload warnings (08-12 analysis). I split the connection pool per workspace and capped agent searches at top_k 8.

Silent graph outage (2026-10-04). When Neo4j stopped, the container still looked healthy. Now /readyz sends RETURN 1 to Neo4j with a 2.5-second limit and reports ok or fail. This failure alone does not return 503, so it does not cause a restart loop. Search answers with 503 and Retry-After 30 after 2 attempts.

> **Scale and evidence.** ~1,470 commits · ~47,000 lines of product code · ~33,600 lines of tests (10-24) · 27 routers · ~1,750 test functions · 13 workspaces (10-24) · Search eval set of 18 cases (query, expected workspace, required evidence, forbidden claims)

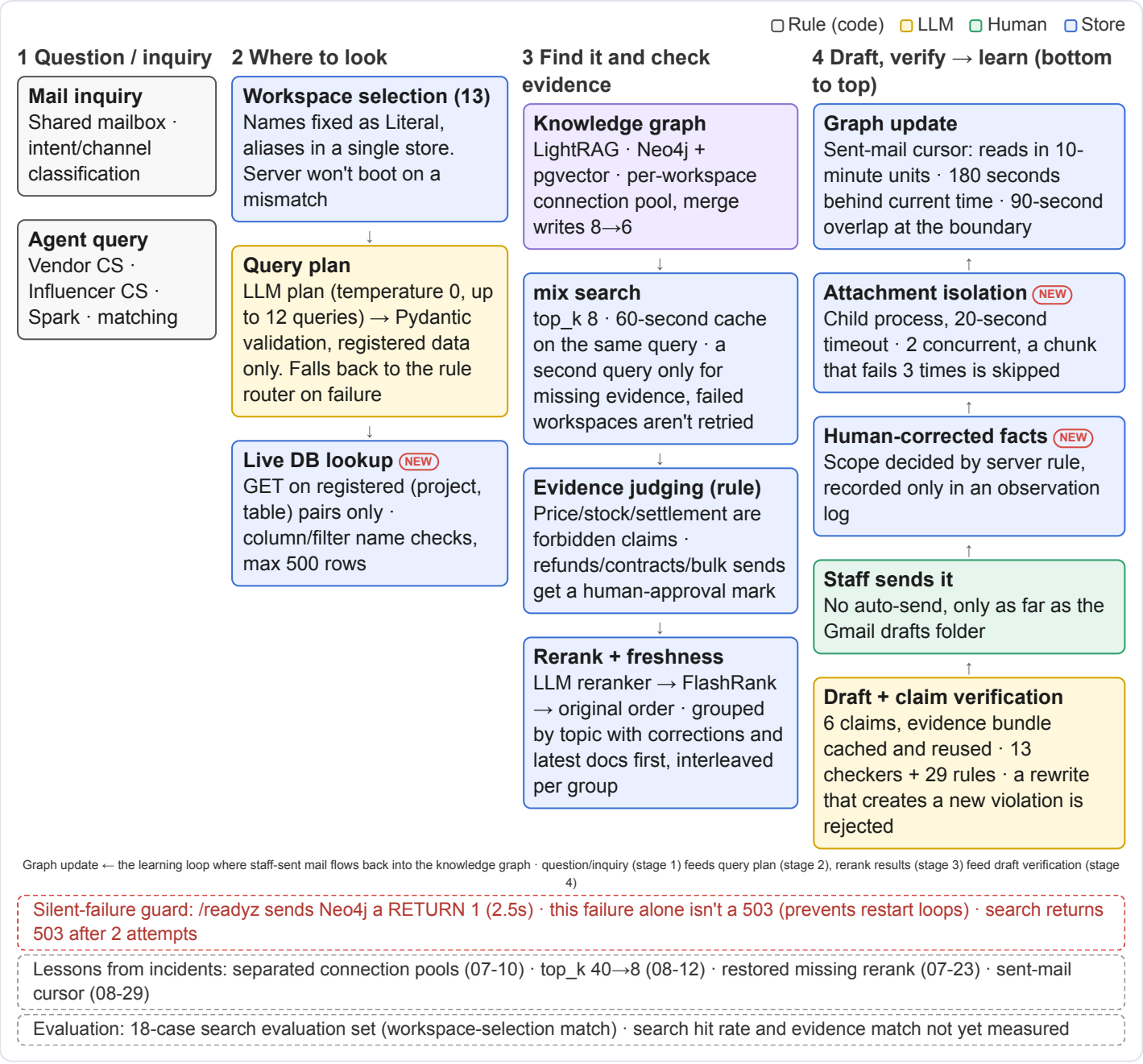
Sent-mail learning missed 48 of 1,943 messages at rollout. I changed cursor handling to overlapping ten-minute windows, lagging current time by 180 seconds. Sent mail, not AI drafts, feeds the knowledge hub. Only server images that pass pytest are deployed.

The evidence cache reuses bundles of up to 150,000 characters for five minutes, with a fallback to the original path. Reranking considers up to 20 documents; search concurrency is limited to 3.

> The search evaluation has 18 cases, but currently checks workspace selection rather than evidence hit rate. The effect of restoring reranking has not been measured. Sent-mail learning uses overlapping windows and a cursor behind current time to account for Gmail indexing lag. I would add evidence-match scoring before making a stronger search-quality claim.

LightRAG Knowledge Hub · Email Agent

Architecture



Technical highlights

1. Validate workspace names at startup. Thirteen canonical names and about fifty aliases resolve to fixed stores. Missing entries or mismatches halt startup.

2. Validate the search plan. Only registered table pairs, GET requests and up to 500 rows are allowed. Invalid plans fall back to the rule router with a recorded reason.

3. Keep reranking available. The fallback sequence is LLM reranker, FlashRank, then original retrieval order. Search concurrency is bounded to reduce graph load.

4. Check distinct failure patterns. Thirteen checkers cover errors seen in real emails, including reversed requests, invented prior messages and proposed prices treated as final.

5. A rewrite cannot add a new falsehood. Reject a newly introduced factual violation even if the total score improves. Drafts with unresolved violations are discarded.

6. Reuse evidence and learn cautiously. A five-minute evidence cache avoids resending the same bundle. Overlapping sent-mail windows account for Gmail lag. Staff corrections remain in an observation log until scope is verified.

Work Internal Work Tool

StyleSeller · Private repositories

Reliable pricing, reviewable catalog candidates and controlled personalized KakaoTalk sends.

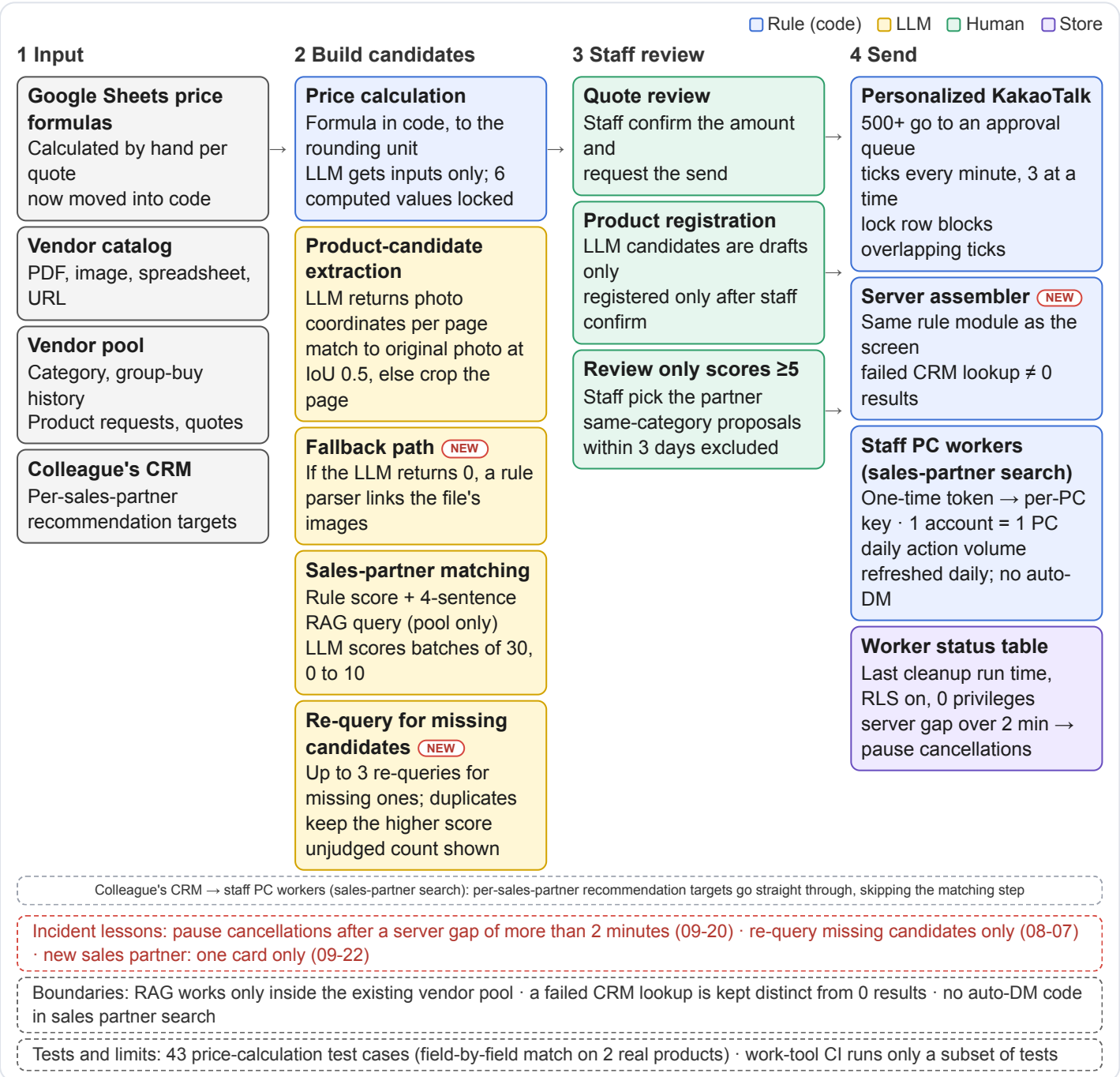
Period	Late March 2026 onward · In production
Role	I built pricing, catalog extraction, partner matching, send automation and PC workers. A colleague's CRM supplies seller-specific recommendations; I built the automation that receives those recipients and sends the messages.
Stack	React · PostgreSQL/PostgREST · Gemini · RAG · KakaoTalk PC workers

Background

Pricing depended on spreadsheet formulas and manual checks. I moved the calculations into code and compared real-product values field by field. The work tool also extracts catalog candidates and finds potential partners, leaving registration and review to staff.

Generating a card for every recipient caused 980 HTTP 500 errors on September 22. I changed the new-seller path to generate the card only once.

Architecture



Internal Work Tool · Pricing, Matching, Personalized KakaoTalk

Design and operations

> **Scale and evidence.** Personalized KakaoTalk sent to 261 people in one day (2026-09-22) · 43 pricing test cases, some compared field by field against sheet values for real products · Seller-finder worker on 9 staff PCs (2026-09-18)

How it works

1. Receive inputs: Spreadsheet rules, supplier PDFs/images/sheets/URLs, the existing partner pool and a colleague's CRM provide the inputs.
2. Calculate and propose: Code computes prices. The LLM returns editable inputs and product candidates. Rules and RAG widen the partner candidate set within the registered pool.
3. Review: Staff confirm prices and catalog items. Matching excludes recent same-category proposals. Sends to 500 or more recipients wait for approval.
4. Run scheduled work: The server assembles sends in small batches. Workers use per-device keys and account assignment. Seller-finder workers have daily limits and contain no auto-DM code.

Incidents and fixes

Missing match candidates (2026-07-15). In 3 of 6 chunks (the batches sent to the LLM separately), the LLM ended its response before judging all candidates. Since 07-16, candidates are collected as a union and then split for querying, and since 08-07 only the missing candidates are re-queried, up to 3 times. The count that still goes unjudged is shown separately in the UI.

All worker PCs shared one key (2026-09-10). Every staff PC worker was sharing a single key. I changed it so each PC gets its own key through a 30-minute one-time install token. The server stores only the key's sha256, and comparison uses timingSafeEqual.

Six runs cancelled at once (2026-09-20). When the server came back after being down for two days, 6 in-progress runs were cancelled at once. The 'worker not seen for 3 minutes' rule read the server gap as workers dropping out. If the interval between cleanup runs exceeds 2 minutes, it is now treated as a server gap and the reference time is reset.

Technical highlights

1. The LLM edits inputs, never computed amounts. Six calculated values are blocked from edits. A margin-only request changes the relevant input; code recalculates prices.

2. Match spreadsheet rounding exactly. 43 tests cover two real-product comparisons (including a KRW 30,000 sample). Both shares sum to total profit, which stays unchanged when their ratio changes.

3. Prefer original product images. Match LLM photo coordinates to embedded PDF images at overlap above 0.5. Exclude backgrounds covering over 80% of the page; crop only if no original matches.

4. Recover from incomplete extraction. A rule-based parser handles empty LLM results and ignores icons below 750 bytes. Missing match scores are retried up to three times; unresolved counts stay visible.

5. RAG only widens the existing pool. Rules and search candidates are combined, but unknown partner IDs are rejected. The LLM scores batches of thirty; staff see scores of five or more.

6. Manual and scheduled sends share rules. Both use the same product-selection and template code. CRM failure differs from an empty result; a new-seller card is generated once rather than per recipient.

7. Scheduled work stays bounded. Ticks assemble small batches, skip when a backlog exists and stop starting outside the send window. Locks prevent overlap; each account is assigned to one PC.

Design choices

Code calculates amounts; the LLM proposes input edits. RAG widens the candidate set only within the existing partner pool. Sends to 500 or more recipients wait in an approval queue.

PCs Staff PC Deployment · Operations

Shipping and operating tools on staff Macs and Windows PCs, then staying responsible after installation.

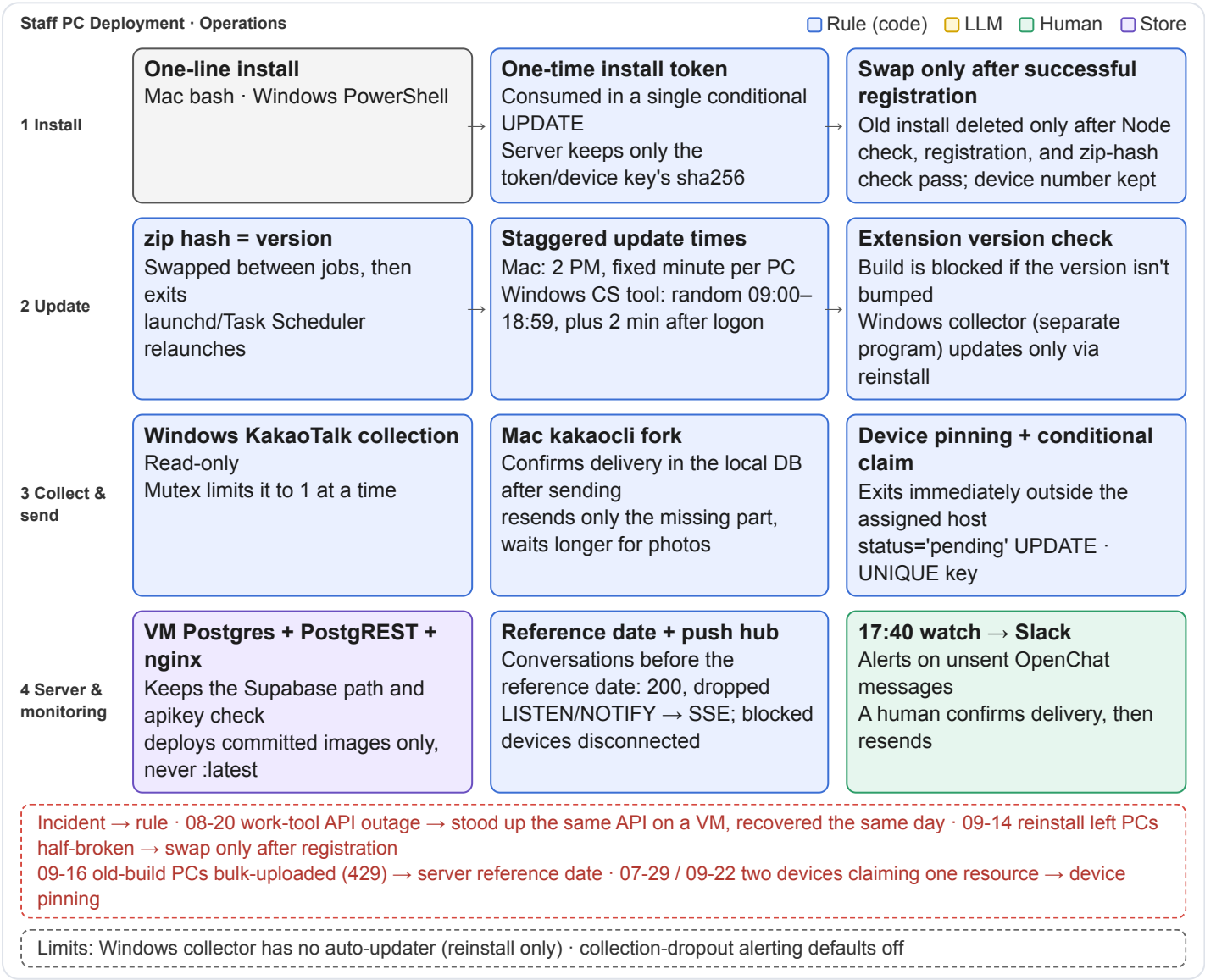
Period	February 2026 onward · In production
Role	I designed installation, authentication and update behavior, built installers and workers, improved the kakaocli fork and Windows collector, and handled releases, monitoring and incident recovery.
Stack	Bash/PowerShell · launchd/Task Scheduler · Swift/Python · Docker · PostgreSQL

Background

These workflows cannot rely on a server-only integration with staff KakaoTalk and Instagram sessions. I built local workers, one-line installers and device authentication, then operated the supporting servers. The deployment snapshot recorded eleven staff PCs, with seller-finder workers on nine PCs.

> **Scale and evidence.** 11 staff PCs (as registered on 2026-09-16), 9 seller-finder worker PCs (09-18) · 4 DBs moved to self-hosting (2026-08-06~08-25) · Full work-tool API outage restored the same day (2026-08-20)

Architecture



How it works

1. Install and register: A one-line command validates the runtime and exchanges an expiring, single-use token for a device key. The server stores SHA-256 hashes.
2. Update safely: The package hash identifies the build. Workers update between jobs and are restarted by the OS scheduler. Update times are staggered by PC.
3. Collect and execute: Jobs are assigned to a device and claimed with one conditional database update. Windows collection is read-only. Mac sends are confirmed in the local database.
4. Observe and recover: A VM serves the API and push hub. Key changes revoke device access; Slack monitoring flags unsent work. Deployments use images tied to a commit.

Incidents and fixes

Full work-tool API outage (2026-08-20). The cloud DB hit its egress quota and every work-tool API stopped. I had nginx on the VM mimic the Supabase paths and apikey header check exactly. The apps changed only their connection settings and a few adapters; the 100-odd createClient calls in app code stayed as they were, and service was back that day.

Reinstall broke healthy PCs (2026-09-14). The installer deleted the old install first and registered after. When it ran with an expired code, the PC was left with no config and no package. Now the old folder is deleted only after the Node check, registration, and zip hash check all pass. The device number stays the same across reinstalls.

Old-version PCs uploaded their whole chat history (2026-09-16). Three Windows PCs sent their backlog of chats all at once. The Sheets API returned rate-limit errors (HTTP 429), and rows for the 2 staff who installed later stayed empty for over an hour. Fixing the client does nothing until the PC picks up the new version. So the server holds a cutoff date, and messages before it are accepted with 200 and then dropped.

Technical highlights

1. Consume install tokens atomically. One conditional UPDATE checks expiry and marks the token used. Two PCs cannot register with the same token; each device has a separate key.

2. Use the package hash as the version. Workers swap packages only between jobs and exit. The OS scheduler restarts them, avoiding competing workers and separate version metadata.

3. Stagger releases by PC. Different update times leave a window to withdraw a bad release. Mac tools and the Windows CS tool auto-update; the separate Windows collector requires reinstalling.

4. Confirm sends in the local database. After a photo-only send was marked successful, I added checks for text and photo arrival and resend only missing parts.

5. Protect device assignment and claims. Unassigned workers exit immediately. Conditional claims and a unique send-date/room key prevent competing work; Windows collection remains read-only.

6. Push revocations before more work. Postgres notifications wake workers over SSE. Key changes trigger re-authentication; a revoked device's connection closes.

7. Deploy the script and image from one commit. The deploy script reloads its target revision. Missing images stop deployment; configuration is rendered before building, and the running code is checked afterwards.

> Counts are dated: eleven PCs registered on September 16 and nine seller-finder workers on September 18. Mac tools and the Windows CS tool update automatically; the separate Windows collector still requires reinstalling. Collection-gap alerts are off by default. A missing release image stops deployment rather than selecting another version; I check the code actually running after deployment.

I retain messages whose timestamps cannot be read rather than silently dropping them. I also preserve the API contract during recovery so client changes stay limited to connection settings and a few adapters.